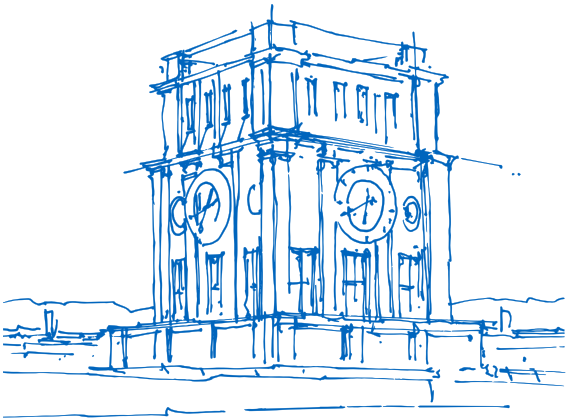
Einführung in die Rechnerarchitektur

Zentralübung #1: Zahlensysteme und Binärarithmetik

Dominic Prinz & Jakob Schäffeler

Lehrstuhl für Rechnerarchitektur & Parallele Systeme
Technische Universität München



Lernziele

- Zahlen zwischen Binär, Dezimal und Hexadezimal umrechnen können
- Grundrechenarten im Binärsystem anwenden können
- Negative Darstellungen (Vorzeichen–Bit, Einer-, Zweierkomplement) kennen und vergleichen können
- Mit Zweierkomplement rechnen können und Überläufe erkennen
- Wertebereiche für n Bit *signed/unsigned* sowie gängige Datentypen bestimmen
- Speichergrößen korrekt einordnen (kB vs. KiB, Zehnerpotenzen vs. Zweierpotenzen).

Zahlen als universelles Datenformat

- Realisierung von Daten im Computer:
  - Basierend rein auf Zahlen
  - Auch Bilder sind Aneinanderreihung von Zahlen
  - Alles wird als Aneinanderreihung von Zahlen repräsentiert
- Strings werden in Zahlen umgewandelt: Oft mit NULL terminiert (C-Strings)
- aber auch explizite Größenangabe zu Beginn möglich (Pascal-Strings)
- beides hat Vor- und Nachteile $\rightarrow$ später mehr dazu

Beispiel für Zeichenformat (ASCII)

- Ordnet jedem Zeichen einen numerischen Wert von 0 bis 127 zu
- Erweiterung von Zeichensatz durch ISO und Unicode (z.B. UTF-8)
- Bildung von Wörtern und Sätzen
- Für Menschen: Aneinanderreihung von Zeichen
- Für Computer: Aneinanderreihung von Zahlwerten

ASCII (1977/1986)

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	NUL 0000 0	SOH 0001 1	STX 0002 2	ETX 0003 3	EOT 0004 4	ENQ 0005 5	ACK 0006 6	BEL 0007 7	BS 0008 8	HT 0009 9	LF 000A 10	VT 000B 11	FF 000C 12	CR 000D 13	SO 000E 14	SI 000F 15
1_	DLE 0010 16	DC1 0011 17	DC2 0012 18	DC3 0013 19	DC4 0014 20	NAK 0015 21	SYN 0016 22	ETB 0017 23	CAN 0018 24	EM 0019 25	SUB 001A 26	ESC 001B 27	FS 001C 28	GS 001D 29	RS 001E 30	US 001F 31
2_	SP 0020 32	! 0021 33	" 0022 34	# 0023 35	$ 0024 36	% 0025 37	& 0026 38	' 0027 39	(0028 40	) 0029 41	* 002A 42	+ 002B 43	, 002C 44	- 002D 45	. 002E 46	/ 002F 47
3_	0 0030 48	1 0031 49	2 0032 50	3 0033 51	4 0034 52	5 0035 53	6 0036 54	7 0037 55	8 0038 56	9 0039 57	: 003A 58	; 003B 59	< 003C 60	= 003D 61	> 003E 62	? 003F 63
4_	@ 0040 64	A 0041 65	B 0042 66	C 0043 67	D 0044 68	E 0045 69	F 0046 70	G 0047 71	H 0048 72	I 0049 73	J 004A 74	K 004B 75	L 004C 76	M 004D 77	N 004E 78	O 004F 79
5_	P 0050 80	Q 0051 81	R 0052 82	S 0053 83	T 0054 84	U 0055 85	V 0056 86	W 0057 87	X 0058 88	Y 0059 89	Z 005A 90	[005B 91	\ 005C 92	] 005D 93	^ 005E 94	_ 005F 95
6_	` 0060 96	a 0061 97	b 0062 98	c 0063 99	d 0064 100	e 0065 101	f 0066 102	g 0067 103	h 0068 104	i 0069 105	j 006A 106	k 006B 107	l 006C 108	m 006D 109	n 006E 110	o 006F 111
7_	p 0070 112	q 0071 113	r 0072 114	s 0073 115	t 0074 116	u 0075 117	v 0076 118	w 0077 119	x 0078 120	y 0079 121	z 007A 122	{ 007B 123	| 007C 124	} 007D 125	~ 007E 126	DEL 007F 127

Letter | Number | Punctuation | Symbol | Other | undefined | Changed from 1963 version

Generische Zahlendarstellung

- Sogenannte *b*-adische Darstellung
- *b* (= Basis) beschreibt Wertebereich jeder Ziffer
- Zahl *a* ist Tupel mit den Elementen a_i (von rechts nach links nummeriert)
- a_i ist die Ziffer an der Stelle *i* und $a_i \in \{0, 1, ..., b-1\}$
- *n* als maximale Stellenanzahl (theoretisch unendlich, praktisch natürlich endlich)
- Berechnung des Zahlenwerts der Zahl $a = a_n a_{n-1} ... a_1 a_0$:

$$Z = \sum_{i=0}^{n} a_i \cdot b^i = a_n \cdot b^n + ... + a_2 \cdot b^2 + a_1 \cdot b^1 + a_0 \cdot b^0$$

Beispiel für 2026

Dezimalsystem ($b = 10$)

- $b = 10$
- $a_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$$2026_{10} = \sum_{i=0}^{3} a_i \cdot 10^i = 2 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 6 \cdot 10^0$$

Binärsystem ($b = 2$)

- $b = 2$
- $a_i \in \{0, 1\}$

$$111_1110_1010_2 = \sum_{i=0}^{10} a_i \cdot 2^i = 1 \cdot 2^{10} + ... + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 2026_{10}$$

Beispiel für 2026

Hexadezimalsystem ($b = 16$)

- $b = 16$
- $a_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$

$$2026_{10} = 111_1110_1010_2 = 7EA_{16}$$

- oft auch mit **0x** präfigiert oder mit **H** suffigiert
- eine vierer Bit-Gruppe (Nibble) für ein hexadezimales Zeichen

Beispiel für 2026

Oktalsystem ($b = 8$)

- $b = 8$
- $a_i \in \{0, 1, 2, 3, 4, 5, 6, 7\}$

$$2026_{10} = 011_111_101_010_2 = 3752_8$$

- oft auch mit **0o** oder nur **0** präfigiert
- eine dreier Bit-Gruppe (Triade) für ein oktales Zeichen

Im Computer

- „Ein Computer versteht nur 0en und 1en“
- Jede binäre Ziffer wird durch ein fehlendes oder ein existierendes Signal beschrieben.
  - 1 = Strom ein.
  - 0 = Strom aus.
- **!** oder auch anders herum (sog. „active low Schaltung“)
- Rechenoperationen erfolgen durch Zusammenschaltung von Signalen und Logikgattern $\rightarrow$ später.

Binäre Arithmetik: Addition

- Ähnlich zu Arithmetik im Dezimalsystem
- Überlauf wird an die nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 001112 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 16_{10}
- Rechnung:

Binäre Arithmetik: Addition

- Ähnlich zu Arithmetik im Dezimalsystem
- Überlauf wird an die nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 001112 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 16_{10}
- Rechnung:

	01001	
+	00111	
+	10	Übertrag
	0	Ergebnis

Binäre Arithmetik: Addition

- Ähnlich zu Arithmetik im Dezimalsystem
- Überlauf wird an die nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 001112 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 16_{10}
- Rechnung:

	01001	
+	00111	
+	110	Übertrag
	00	Ergebnis

Binäre Arithmetik: Addition

- Ähnlich zu Arithmetik im Dezimalsystem
- Überlauf wird an die nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 001112 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 16_{10}
- Rechnung:

	01001	
+	00111	
+	1110	Übertrag
	000	Ergebnis

Binäre Arithmetik: Addition

- Ähnlich zu Arithmetik im Dezimalsystem
- Überlauf wird an die nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 001112 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 16_{10}
- Rechnung:

	01001	
+	00111	
+	11110	Übertrag
	0000	Ergebnis

Binäre Arithmetik: Addition

- Ähnlich zu Arithmetik im Dezimalsystem
- Überlauf wird an die nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 001112 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 16_{10}
- Rechnung:

	01001	
+	00111	
+	11110	Übertrag
	10000	Ergebnis

Binäre Arithmetik: Subtraktion

- auch ähnlich zu Arithmetik im Dezimalsystem?
- Versuch: Überlauf wird an nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 00111_2 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 2_{10}
- Rechnung:

Binäre Arithmetik: Subtraktion

- auch ähnlich zu Arithmetik im Dezimalsystem?
- Versuch: Überlauf wird an nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 00111_2 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 2_{10}
- Rechnung:

	01001	
–	00111	
–	00	Übertrag
	0	Ergebnis

Binäre Arithmetik: Subtraktion

- auch ähnlich zu Arithmetik im Dezimalsystem?
- Versuch: Überlauf wird an nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 00111_2 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 2_{10}
- Rechnung:

	01001	
–	00111	
–	100	Übertrag
	10	Ergebnis

Binäre Arithmetik: Subtraktion

- auch ähnlich zu Arithmetik im Dezimalsystem?
- Versuch: Überlauf wird an nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 00111_2 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 2_{10}
- Rechnung:

	01001	
–	00111	
–	1100	Übertrag
	010	Ergebnis

Binäre Arithmetik: Subtraktion

- auch ähnlich zu Arithmetik im Dezimalsystem?
- Versuch: Überlauf wird an nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 00111_2 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 2_{10}
- Rechnung:

	01001	
–	00111	
–	01100	Übertrag
	0010	Ergebnis

Binäre Arithmetik: Subtraktion

- auch ähnlich zu Arithmetik im Dezimalsystem?
- Versuch: Überlauf wird an nächste Stelle „weitergereicht“
- Beispiel:
  - System mit max. 5 Ziffern.
  - $a = 01001_2 = 1 \cdot 8_{10} + 0 \cdot 4_{10} + 0 \cdot 2_{10} + 1 \cdot 1_{10} = 9_{10}$
  - $b = 00111_2 = 0 \cdot 8_{10} + 1 \cdot 4_{10} + 1 \cdot 2_{10} + 1 \cdot 1_{10} = 7_{10}$
  - Ergebnis: 2_{10}
- Rechnung:

	01001	
–	00111	
–	01100	Übertrag
	00010	Ergebnis

Binäre Arithmetik: Subtraktion

- Fazit: Funktioniert, aber Problem bei $7 - 9$

	00111	
–	01001	
–	00	Übertrag
	0	Ergebnis

Binäre Arithmetik: Subtraktion

- Fazit: Funktioniert, aber Problem bei $7 - 9$

	00111	
–	01001	
–	000	Übertrag
	10	Ergebnis

Binäre Arithmetik: Subtraktion

- Fazit: Funktioniert, aber Problem bei $7 - 9$

	00111	
–	01001	
–	0000	Übertrag
	110	Ergebnis

Binäre Arithmetik: Subtraktion

- Fazit: Funktioniert, aber Problem bei $7 - 9$

	00111	
–	01001	
–	10000	Übertrag
	1110	Ergebnis

Binäre Arithmetik: Subtraktion

- Fazit: Funktioniert, aber Problem bei $7 - 9$

	00111	
–	01001	
–	110000	Übertrag
	11110	Ergebnis

Binäre Arithmetik: Subtraktion

- Fazit: Funktioniert, aber Problem bei $7 - 9$

	00111	
–	01001	
–	110000	Übertrag
	11110	Ergebnis

- unendlicher Übertrag?
- $\Rightarrow$ Suche nach eleganterer Möglichkeit
- bei allen folgenden Varianten müssen sich Kommunikationspartner im Vorhinein auf eine Darstellung einigen

Variante: Das Sign-Bit

- Verwende höchstwertiges Bit, um Vorzeichen darzustellen.
- Beispiel: Darstellung einer Zahl mit 8 Bits
  - $+7_{10}$ = 0000_0111_2
  - -7_{10} = 1000_0111_2
- Vorteile:
  - trivial zu realisieren.
  - Vorzeichen durch erstes Bit repräsentiert $\rightarrow$ direkt ablesbar
- Nachteile:
  - Spezialbehandlung für Addition und Subtraktion.
  - Reduktion der Anzahl an darstellbaren Werten (zwei Darstellungen für die Zahl 0).

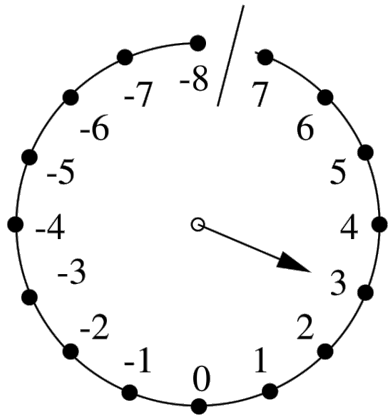
Variante: Das Zweierkomplement

- Schritt 1: Einerkomplement
  - Bitkomplement: Invertierung jedes einzelnen Bits
  - Beispiel: Negation von Zahl 7_{10}: $0000\ 0111_2 \rightarrow 1111\ 1000_2$
- Schritt 2: Addition von „1“
  - $-7_{10} = 1111\ 1001_2 = -a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i = -1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^0$
- Vorteile:
  - siehe folgende Folien
  - Vorzeichen durch erstes Bit repräsentiert $\rightarrow$ negative Zahl direkt erkennbar, aber Zahlenwert nicht
- Nachteile:
  - Zahl nicht direkt „ablesbar“ (außerdem: Breite muss bekannt sein)
  - kein Gegenstück für kleinste negative Zahl im positiven Zahlenbereich

Variante: Das Zweierkomplement

Beispiel mit 4 Bits:

$$3_{10} = 0011_2$$
$$2_{10} = 0010_2$$
$$1_{10} = 0001_2$$
$$0_{10} = 0000_2$$
$$-1_{10} = 1110_2 + 1_2 = 1111_2$$
$$-2_{10} = 1101_2 + 1_2 = 1110_2$$
$$\ldots$$
$$-7_{10} = 1000_2 + 1_2 = 1001_2$$
$$-8_{10} = 0111_2 + 1_2 = 1000_2$$



- Ausblick: Das Überlaufen zu negativen Werten verursacht oft Probleme, wenn man sich dessen nicht bewusst ist

Eigenschaften des Zweierkomplements

- Wertebereich bei n Bits: $\left[-2^{n-1}, 2^{n-1}-1\right]$
- Bitweise Darstellung von
  - positivem Zahlenbereich wie gewohnt
  - negativem Zahlenbereich mit Zweierkomplement
- Vorteile:
  - identische Addition: $(-1) + (-2) = 1111_2 + 1110_2$

	1111	
+	1110	
+	11100	Übertrag
	1101	Ergebnis

    Und $1101_2 = -(0010_2 + 1) = -0011_2 = -3_{10}$
  - kein (unnötiger) Verlust einer Ziffer

Subtraktion mit Zweierkomplement

- Beispiel:
  - System mit max. 5 Ziffern
  - $a = 01001_2 = 9_{10}$
  - $b = 00111_2 = 7_{10}$
  - Aufgabe: $9 - 7$
- Schitte:
  1. Einerkomplement
     - $00111_2 \rightarrow 11000_2$
  2. Addition von 1
  3. Addition der Zahlen

Subtraktion mit Zweierkomplement

- Beispiel:
  - System mit max. 5 Ziffern
  - $a = 01001_2 = 9_{10}$
  - $b = 00111_2 = 7_{10}$
  - Aufgabe: $9 - 7$
- Schitte:
  1. Einerkomplement
     - $00111_2 \rightarrow 11000_2$
  2. Addition von 1
     - $11000_2 + 1_2 = 11001_2$
  3. Addition der Zahlen

Subtraktion mit Zweierkomplement

- Beispiel:
  - System mit max. 5 Ziffern
  - $a = 01001_2 = 9_{10}$
  - $b = 00111_2 = 7_{10}$
  - Aufgabe: $9 - 7$
- Schitte:
  1. Einerkomplement
  2. Addition von 1
  3. Addition der Zahlen

	01001	
+	11001	
+	10010	Übertrag
	00010	Ergebnis

Über/Unterläufe

- Überschreitung/Unterschreitung des darstellbaren Wertebereiches
- Beispiel:
  - $a = 0111_2 = 7_{10}$
  - $b = 0001_2 = 1_{10}$
  - $a + b = 1000_2 = -8_{10}$
- $\rightarrow$ Falls nötig auf Über/Unterläufe testen
- Bspw. in `gcc`: `__builtin_add_overflow()`

Multiplikation

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel: $7 \cdot 5 = 0111_2 \cdot 0101_2$
- Idee: $7 \cdot 5 = 1 \cdot 7 + 4 \cdot 7$

Multiplikation

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel: $7 \cdot 5 = 0111_2 \cdot 0101_2$
- Idee: $7 \cdot 5 = 1 \cdot 7 + 4 \cdot 7$

	0111 · 0101	
	0111	
+	0000	
+	0111	
+	111000	Übertrag
	100011	Ergebnis

Multiplikation

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel: $7 \cdot 5 = 0111_2 \cdot 0101_2$
- Idee: $7 \cdot 5 = 1 \cdot 7 + 4 \cdot 7$

	$0111 \cdot 0101$	
	0111	
+	0000	
+	0111	
+	111000	Übertrag
	100011	Ergebnis

- $100101_2 = 32_{10} + 2_{10} + 1_{10} = 35_{10}$

Division

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel im Dezimalsystem: $51/3$

Division

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel im Dezimalsystem: $51/3$

	51 / 3	= 1
	5	
–	3	
	2	

Division

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel im Dezimalsystem: $51/3 = 17$

```
    51 / 3   = 17
  ---------
    5
–   3
  ---------
    21
–   21
  ---------
     0
```

Division

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel im Dezimalsystem: $51/3 = 17$
- Beispiel in Binärsystem: $21/3 = 10101_2/11_2$

Division

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel im Dezimalsystem: $51/3 = 17$
- Beispiel in Binärsystem: $21/3 = 10101_2/11_2$

```
      10101 / 11    = 1
    -----------------
      101
  -    11
    -----------------
      010
```

Division

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel im Dezimalsystem: $51/3 = 17$
- Beispiel in Binärsystem: $21/3 = 10101_2/11_2$

```
      10101 / 11    = 11
-----------------------
      101
  -    11
-----------------------
      0100
  -    11
-----------------------
       001
```

Division

- Ähnlich zu Arithmetik im Dezimalsystem.
- Beispiel im Dezimalsystem: $51/3 = 17$
- Beispiel in Binärsystem: $21/3 = 10101_2/11_2$

```
      10101 / 11    = 111
-----------------------
      101
-     11
-----------------------
      0100
-     11
-----------------------
       0011
-      11
-----------------------
         00
```

Speichergrößen

- „Kilo-“, „Mega-“, „Giga-“ usw. sind sog. SI-Präfixe
- stehen für Zehnerpotenzen: Kilo = 10^3 = 1000, Mega = 10^6 usw.
- In der Informatik sind allerdings Zweierpotenzen deutlich üblicher/sinnvoller
- Deshalb gibt es daran angelehnte sog. Binärpräfixe (auch IEC-Präfixe):
- Beispiel: Kibi = 2^{10} = 1024 $\neq$ 1000 = 1^3 = Kilo
- zu erkennen am „bi“ für **bi**när
- Auch ihre Abkürzungen sind anders:
  - SI: K-, M-, G-, T-, ...
  - IEC: Ki-, Mi-, Gi-, Ti-, ...
- Aufpassen! Deshalb zeigt ein 16 GB USB-Stick auch nicht 16 GiB an.

Gängige ganzzahlige C-Datentypen

- RV32IM benutzt die ILP32-ABI, d.h. `int`, `long` und Zeiger (pointer) sind 32-Bit breit
- Die M-Erweiterung ändert das Datenmodell nicht
- Laut ABI ist `char` vorzeichenlos

C-Typ	Bits	Vorzeichen	min.	max.
`(unsigned) char`	8	nein	0	$2^8 - 1$
`signed char`	8	ja	-2^7	$2^7 - 1$
`unsigned short`	16	nein	0	$2^{16} - 1$
`(signed) short`	16	ja	-2^{15}	$2^{15} - 1$
`unsigned int, unsigned long`	32	nein	0	$2^{32} - 1$
`(signed) int, (signed) long`	32	ja	-2^{31}	$2^{31} - 1$
`unsigned long long`	64	nein	0	$2^{64} - 1$
`(signed) long long`	64	ja	-2^{63}	$2^{63} - 1$

Quiz

https://partici.fi/93236618

